SEOULTECH
Seoul National University of Science & Technology

Web Programming

Full-Stack JavaScript RESTful Web Application

1. OVERVIEW

Each team builds a complete full-stack web application of its own choice. Teams have freedom in selecting the application domain, frontend technology, and database, but must implement a documented REST API, a working responsive frontend, authentication, automated testing, and continuous integration.

1.1 Learning objectives

- Design and implement a documented REST API following resource-oriented conventions.
- Build a responsive frontend that consumes the REST API.
- Apply authentication and basic security to a user-facing web application.
- Write unit and API-level tests covering meaningful parts of the backend.
- Configure continuous integration to run tests automatically on every push.
- Collaborate effectively in a team using Git and a shared repository.

2. TEAMS AND TIMELINE

2.1 Team composition

- Teams of 3 to 4 students. The course will form 7 teams in total: five teams of 4 and two teams of 3.
- Team formation deadline: Wednesday, May 13, 2026.
- Each team appoints one team lead, responsible for communication with the instructor.
- All team members must commit to the shared Git repository under their own accounts. Lack of individual commit history is treated as evidence of non-participation.

2.2 Key dates

Date	Milestone	Description
Wed, May 13	Team formation	Teams must be formed and registered with the instructor. Initial application idea identified in one or two sentences.
Fri, May 22	Project proposal	Submit proposal slides (9 slides, structure defined in Section 4). No in-class presentation; submission only.
Weekly, May 22 to Jun 5	In-class feedback and review	Time is reserved each week for teams to ask questions, demonstrate progress, and receive feedback from the instructor.
Tue, Jun 9	Final delivery	Deployed application live at the team's URL (if you only deployed your application locally, provide instructions on how to run the application locally). Repository tagged for evaluation. Final presentation slides prepared.
Jun 10 - Jun 12	Live project evaluation	Each team attends a 30-minute scheduled session in the instructor's office. Schedule announced after team formation.

SEOULTECH
Seoul National University of Science & Technology

3. APPLICATION REQUIREMENTS

The application is the team's own product. Feature scope and visual design are part of the proposal. The requirements below define the minimum technical points that every project must meet.

3.1 Architecture

- Backend implemented in Node.js with Express, exposing a RESTful HTTP API.
- Database: free choice (SQLite, PostgreSQL, MongoDB, or others). The choice must be justified in the proposal. Use of an in-memory store as the only persistence mechanism is not acceptable.
- Frontend: free choice of technology. Server-side templating, vanilla JavaScript, React, Vue, Svelte, or other frameworks are all permitted. Teams must be able to explain their choice during the live evaluation.
- The frontend must be responsive: the application must render and function correctly on both desktop screens and mobile-sized viewports.

3.2 REST API

- All data interactions go through the REST API. The frontend is a client of the same API.
- Use HTTP methods consistently: GET for retrieval, POST for creation, PUT or PATCH for updates, DELETE for removal.
- Use status codes consistently: 2xx for success, 4xx for client errors, 5xx for server errors.
- All endpoints return JSON. Error responses include at minimum an error code and a human-readable message.
- REST API documentation generated with Swagger / OpenAPI is required, covering authentication endpoints and at least the main resource endpoints.

3.3 Authentication and security

- End users of the application authenticate.
- At least one route beyond login itself must require authentication, and the protection must be enforced server-side.
- Passwords are stored hashed. Plaintext passwords in the database or in logs are an automatic deduction.
- Basic input validation on write endpoints, and documented mitigation of at least one common vulnerability (XSS, CSRF, SQL injection, or broken access control).

3.4 Testing

- Unit tests for at least two non-trivial backend modules (e.g., a controller function, a validator, a utility).
- API tests covering at minimum: one authenticated route, one error case (such as 401, 404, or 400), and one successful main-flow endpoint.
- Tests must be runnable via a single command (e.g., npm test).

3.5 Continuous integration

- A GitHub Actions workflow that runs the test suite automatically on every push to the main branch.
- The workflow status badge included in the repository README.
- Continuous deployment (CD) is optional. Teams that implement it should highlight it during the final evaluation.

3.6 Deployment

- The application might be deployed to a publicly accessible URL by the final delivery date.
- If you only deployed your application locally, provide instructions on how to run the application locally
- The deployed URL must be reachable from the instructor's machine during the live evaluation.

3.7 Documentation

- README covering: project description, technology stack, setup and run instructions, environment variables, test instructions, deployed URL.
- OpenAPI specification file (YAML or JSON) at a documented path in the repository, served by the deployed application at a documented route (e.g., /api/docs).

4. PROPOSAL SLIDES (DUE MAY 22)

The proposal is delivered as a deck of slides only, no written document, no in-class presentation. Minimum 9 slides, maximum 12 (considering only content), ideally one slide per item below. The proposal forces early clarity about what will be built and how. Use this list of items as the slide order and structure.

Slide 1: Application identity

- Application name.
- Team members and team lead.
- One-paragraph description: what the application does, who it is for, and why it matters.

Example:

TableNow — a restaurant reservation app for university-area restaurants.

Team: A. Park, B. Lee, C. Kim, D. Chen (lead).

TableNow lets students reserve tables at participating restaurants near campus, view real-time availability, and cancel reservations from any device. Restaurant owners use it to manage their seating without phone calls.

Slide 2: Application overview

- A high-level description of the application: the problem it solves, the type of users, the main interaction flow.

Example:

Many small restaurants near campus do not have online reservation systems and rely on phone calls during peak hours. Customers cannot see availability without calling, and owners often miss calls during busy periods. TableNow provides a shared platform: restaurants publish their available time slots, customers book directly, and both sides receive confirmation. The main flow is: customer browses restaurants, selects a date and time slot, confirms reservation; restaurant owner sees the reservation appear in their dashboard.

SEOULTECH
Seoul National University of Science & Technology

Slide 3 — Core features with user stories

- List the core features of the application. These are features the team commits to delivering.
- Each core feature is accompanied by a user story in the format: As a [type of user], I want [to do something] so that [I get some benefit].
- At least 4 core features. Each one should be small enough to describe in a single line.

Example:

1. Browse restaurants

As a customer, I want to see a list of participating restaurants so that I can choose where to eat.

2. View available time slots

As a customer, I want to see available time slots for a chosen restaurant and date so that I can pick a time that fits my schedule.

3. Make a reservation

As a customer, I want to reserve a specific time slot so that the table is held for me.

4. Cancel a reservation

As a customer, I want to cancel my reservation so that I can free up the table if my plans change.

5. View today's reservations (owner)

As a restaurant owner, I want to see all reservations for today so that I can prepare the dining room.

Slide 4: Stretch features

- A short list of features the team would like to implement if time permits, but does not commit to.
- No user stories required for stretch features. a one-line description is enough.
- This slide forces explicit prioritization. Implementing stretch features adds value only after all core features are working.

Example:

- Email confirmation when a reservation is made or canceled.

- Filter restaurants by cuisine type.

- Restaurant ratings and reviews.

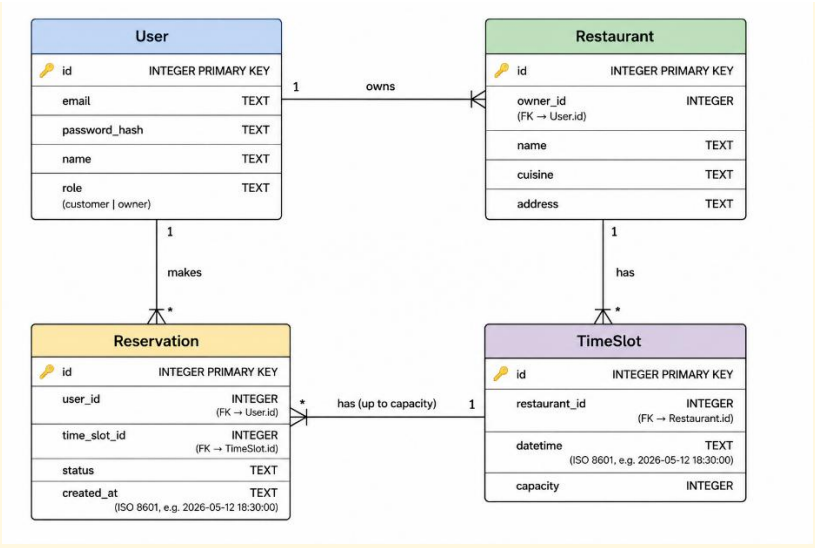
- Reservation history for logged-in customers.

- Owner dashboard analytics (busiest hours, no-show rate).

Slide 5: Data model

- List the main entities of the application (typically 3 to 6).
- For each entity, list its key fields. Mark the primary key.
- Indicate relationships between entities (one-to-many, many-to-many).
- A simple diagram is preferred over prose. Hand-drawn or made with any tool. Clarity matters.

Example:



Relationships:

- One User (owner) has many Restaurants.

- One Restaurant has many TimeSlots.

- One User (customer) has many Reservations.

- One TimeSlot has many Reservations (up to capacity).

Slide 6: REST API design

- A table of the planned API endpoints.
- For each endpoint, indicate: HTTP method, path, purpose, and whether authentication is required.
- Aim for 6 to 12 endpoints. More than 15 is usually a sign of over-scoping.
- Do not include implementation details (database queries, internal logic) — only the public API contract.

Example:

GET	*/api/restaurants*	*List all restaurants*	*Public*
GET	*/api/restaurants/:id*	*Get one restaurant*	*Public*
GET	*/api/restaurants/:id/slots*	*List available time slots*	*Public*

SEOULTECH
Seoul National University of Science & Technology

POST /api/reservations Create a reservation Auth

GET /api/reservations/mine List my reservations Auth

DELETE /api/reservations/:id Cancel my reservation Auth

POST /api/auth/register Register a new user Public

POST /api/auth/login Log in Public

GET /api/owner/reservations Today's reservations for my place Auth (owner)

Slide 7: Technology choices

- Frontend technology: framework or approach chosen, with a one-line rationale.
- Database: which database engine, with a one-line rationale.
- Deployment target: where the application will be hosted (Optional).
- Any other relevant libraries or tools (template engine, ORM, authentication library).

Example:

Frontend: React. Chosen because the team has prior experience and the app has interactive components (calendar, reservation flow) that benefit from a SPA model.

Database: SQLite. Chosen for simplicity in development; one entity relationship is sufficient for our data volume.

Deployment: Render (free tier) for the backend; the frontend served as static files from the same backend.

Additional libraries: express-session for sessions, bcrypt for password hashing, swagger-jsdoc + swagger-ui-express for API documentation.

Slide 8: UI sketches

- Low-fidelity wireframes of the 3 to 5 main screens of the application.
- Hand-drawn photos, Figma, Excalidraw, or any other tool are all acceptable. Visual polish is not required.
- What is evaluated: that the team has thought concretely about layout, navigation, and what users will see and click.
- Indicate clearly which screens are shown (e.g., "Home / restaurant list", "Restaurant detail", "My reservations").

Slide 9: Work plan

- Division of work across team members: which member is responsible for which area (e.g., backend API, frontend UI, database, testing, documentation).

- Weekly milestones from May 22 to June 5: what the team commits to having working at the end of each week.
- This is a plan, not a contract, it can be revised. But it must be specific enough to be actionable.

Example:

Responsibilities:

- A. Park: Backend API (restaurants, time slots).

- B. Lee: Backend API (reservations, authentication).

- C. Kim: Frontend (customer screens).

- D. Chen: Frontend (owner screens) + deployment + CI.

Weekly milestones:

- Week of May 22-26: Backend skeleton, database schema, GET endpoints working.

- Week of May 27-31: Authentication, POST/DELETE endpoints, frontend skeleton.

- Week of Jun 1-5: Full integration, responsive design, testing, deployment.

5. FINAL DELIVERY AND LIVE EVALUATION

5.1 Final delivery (June 9)

By Tuesday, June 9, end of day, each team must have:

- The application deployed and reachable at a public URL (optional).
- The Git repository tagged or with a clear commit identifying the version to be evaluated.
- The final presentation slides prepared (structure in Section 5.3).
- README and OpenAPI documentation in place.

5.2 Live evaluation (June 10 — June 12)

Each team attends a 30-minute scheduled session in the instructor's office. The schedule will be announced after team formation. The full team must attend.

The 30 minutes are structured as follows:

Duration	Activity
8 min	Team presentation. Concise overview of the project using the final presentation slides. The team chooses how to divide the speaking among members.
5 min	Live demonstration of the deployed application, including a walk-through of at least one authenticated flow.
~20 min	Direct questions from the instructor. Questions cover: code structure, design decisions, security choices, REST API design, responsive design behavior, testing approach, and a live code modification task. Questions may be directed at specific team members.

The live code modification task is small, typically adding a new endpoint, fixing a small bug, or modifying behavior in response to a prompt. Teams should be prepared to open their editor and make changes during the session. This is the primary mechanism for verifying that team members understand the code they submitted.

5.3 Final presentation slides

The final presentation slides cover the same topics as the proposal, but report on what was actually built, and add two new topics (authentication and testing) that were not part of the proposal. One slide per item, 11 slides total.

Slide	Content
1	Application identity. Name, team, one-paragraph description.
2	Application overview. What was built, who it is for, the main user flow.
3	Features implemented. Final list of features actually delivered, with user stories for the core ones. Indicate which proposed features were dropped and why.
4	Stretch features delivered (if any). Short list of additional features completed beyond the core.
5	Data model. Final entities, fields, and relationships as implemented.
6	REST API. Final list of endpoints with method, path, and authentication requirement. Mention the URL of the live OpenAPI documentation.
7	Technology stack. Frontend, database, deployment, libraries actually used. Briefly mention any changes from the proposal and why.
8	UI walkthrough. Screenshots of the main screens, including both desktop and mobile views to demonstrate responsive design.
9	Authentication and authorization. Mechanism used (sessions / tokens), how passwords are stored, which routes are protected, and the security measures implemented (e.g., input validation, mitigation of one vulnerability).
10	Testing and CI. What was tested (unit and API), how the tests are organized, and a screenshot of the CI workflow running on GitHub. Mention if CD is also configured.
11	Work breakdown. Final division of work across team members and a brief reflection on what went well and what was difficult.

6. EVALUATION

The project is worth 30% of the final course grade, distributed as follows:

Component	Weight	What is evaluated
Proposal slides (May 21)	20%	Clarity and feasibility of the application idea. Quality of the data model. Soundness of the REST API design. Appropriateness of the technology choices. Concreteness of the work plan.
Live evaluation (Jun 9-10)	80%	The application itself: feature completeness, REST API correctness, authentication, security, code quality, responsive design, testing, CI,

		documentation, and individual understanding verified through direct questioning and live code modification.

6.1 Individual grading within teams

The live evaluation produces a team grade. Individual grades may diverge from the team grade based on: individual contributions visible in the Git commit history, individual performance during the live questioning, and ability to explain and modify code attributed to that student.

A student unable to explain code they committed, or unable to perform a reasonable code modification during the live evaluation, may receive an individual grade lower than the team grade, regardless of overall project quality.

7. ACADEMIC INTEGRITY AND AI USE

Use of AI assistants during the project is permitted but must be disclosed in the README. The live evaluation is the mechanism by which individual understanding is verified. Submitting code that a student cannot read, modify, or explain is treated as a failure of the individual component of the evaluation, even if the team product is functional.

Plagiarism between teams, including reuse of another team's code or proposal content, is treated as a violation of academic integrity and applies to all members of the involved teams.